

SYSTEMY CYBER-FIZYCZNE Z UCZENIEM

ZAJĘCIA PROJEKTOWE

Model predykcyjny do wyceny mieszkań z interfejsem webowym

Jakub Grych 272583
Kateryna Fedoruk 272609
Marian Pawłowski 272591

08 Grudnia, 2025



Politechnika
Wrocławska

Informatyczne systemy automatyki
Politechnika Wrocławska
Polska

Prowadzący kursu dr hab. inż. Andrzej Pawłowski

1 Wstęp

Celem projektu było stworzenie modelu predykcyjnego pozwalającego na przewidzenie z dużą dozą pewności potencjalnej mieszkanie z użyciem wprowadzanych danych odpowiadających danym dostępnym w serwisach aukcyjnych nieruchomości, wraz z interfejsem webowym umożliwiającym korzystanie z modelu.

Rozwiązanie miało umożliwić wycenę, oraz ewentualną ocenę rzeczywistej ceny mieszkania aby określić czy:

- Cena mieszkania jest odpowiednia (cena nieodbiegająca znacząco od predykcji)
- Oferta jest korzystna cenowo (cena znacząco poniżej przewidzianej wartości)
- Oferta jest nieopłacalna (cena znacznie powyżej przewidzianej wartości)

W celu pozyskania danych do rozpatrzenia wybrano dwie opcje:

1. Web-scraping serwisu Otodom w celu uzyskania odpowiedniej ilości rzeczywistych ogłoszeń do wytrenowania modelu. Zbudowano program „crawler”, który umożliwił sprawne zebranie dużej ilości danych, jednocześnie będący odporny na zabezpieczenia strony przed tego typu programami.
2. Użycie gotowych zestawów danych dostępnych w serwisie [Kaggle.com](https://www.kaggle.com/).

2 Proces zbierania danych

W projekcie głównym założeniem była możliwość trenowania oraz oceny modelu przewidującego ceny mieszkań na podstawie ich cech opisowych. W tym celu pozyskano rzeczywistych danych, odzwierciedlających aktualne warunki rynkowe. W ramach projektu zebrano duży zbiór danych bezpośrednio z największego polskiego serwisu ogłoszeniowego `Otodom.pl`.

Aby pozyskać dane, konieczne było zastosowanie web scrapingu, czyli automatycznego pobierania treści stron. W projekcie użyto bibliotek `Playwright` oraz `BeautifulSoup`, które pozwalają symulować zachowanie prawdziwej przeglądarki i analizować strukturę HTML. Scrapowanie jest jednak zadaniem bardziej złożonym niż zwykle pobranie strony, ponieważ Otodom, podobnie jak większość dużych portali, stosuje różnego rodzaju mechanizmy zabezpieczające przed botami. Obejmują one m.in. ograniczenia liczby żądań, filtrowanie powtarzalnych zapytań, a w skrajnych przypadkach blokowanie adresu IP.

W trakcie prac konieczne było rozwiązanie kilku istotnych wyzwań:

- Ominięcie mechanizmów antybotowych, takich jak kontrola szybkości odpytywania serwera, analiza nagłówków HTTP czy sprawdzanie obecności cech typowych dla botów.
- Zachowanie rozsądnego tempa pozyskiwania danych: zbyt szybkie zapytania mogły prowadzić do tymczasowych blokad, zaś zbyt wolne znacznie wydłużały czas pobierania danych.
- Stabilność scrapera, który musiał radzić sobie z błędami sieciowymi, timeoutami oraz niestabilnością struktury strony (częste zmiany layoutu).
- Optymalizacja prędkości, konieczna ze względu na dużą liczbę ofert (kilkudziesięciu tysięcy), co wymagało przejścia z trybu synchronicznego na asynchroniczny, pozwalający pobierać wiele stron jednocześnie.

Trzeba wspomnieć, że scrapowanie wiąże się z pewnymi aspektami prawnymi. Serwisy internetowe mogą zabraniać automatycznego pobierania treści w regulaminie lub ograniczać je poprzez plik `robots.txt`. Dane kontaktowe np. numery telefonów to dane osobowe, które należy przetwarzać zgodnie z przepisami o ochronie danych. W projekcie pozyskiwano i przetwarzano wyłącznie dane niezbędne do analizy, a surowe HTML były przechowywane wyłącznie w lokalnym środowisku.

2.1 Podejście i implementacja

Proces pozyskiwania danych został zrealizowany w dwóch etapach: pierwszym prototypowym, bazującym na rozwiązaniu synchronicznym, oraz finalnym, zoptymalizowanym podejściu asynchronicznym. Ewolucja scrapera wynikała z konieczności zwiększenia wydajności oraz poprawy odporności na mechanizmy antybotowe stosowane przez serwis Otodom. Poniżej omówiono kolejne kroki, rozwiązania techniczne oraz przedstawiono kluczowe fragmenty kodu.

2.1.1 Pierwsza wersja scrapera – podejście synchroniczne

Pierwszy prototyp został stworzony w oparciu o bibliotekę Playwright działającą w trybie synchronicznym. Rozwiązanie to było najprostsze do zaimplementowania oraz dawało pełną stabilność działania. Każda oferta była pobierana osobno — dopiero po zakończeniu jednego żądania następowało kolejne.

```
status, html = goto_with_retries(page, offer_url, timeout=30000)
if html is None:
    continue

fname = re.sub(r"[^a-zA-Z0-9]", "_", offer_url.strip("/"))[-120:] + ".html"
(raw_html_dir / fname).write_text(html, encoding="utf-8")

time.sleep(random.uniform(1.0, 2.5))
```

To podejście było łatwe w debugowaniu, dobrze radziło sobie z prostymi blokadami i zapewniało stabilność i przewidywalność działania, ale przy dużej skali ofert potrzebnych do wytrenowania scrapera, czas działania takiego scrapera wynosił parędziesiąt godzin, co było nieakceptowalne dla tego projektu.

2.2 Architektūrę asynchroniczna

Aby skrócić czas pozyskiwania danych, konieczne było stworzenie rozwiązania wykorzystującego asynchroniczne wywołania Playwrighta, dzięki czemu wiele stron ofert mogło być pobieranych jednocześnie. W finalnej wersji scrapera zastosowano:

- `asyncio` — do równoległego uruchamiania zadań
- `async_playwright` — asynchroniczna wersja Playwrighta
- semaforey (`asyncio.Semaphore`) — sterowanie maksymalną równoległością
- losowe User-Agenty i stealth mode — utrudnianie wykrycia bota
- adaptacyjny system pauz — ograniczający tempo zapytań

Fragment kodu inicjujący workerów:

```
sem = asyncio.Semaphore(args.concurrency)

async def worker(offer_url):
    async with sem:
        context = await browser.new_context(
            user_agent=random.choice(UA_POOL),
            locale="pl-PL"
        )
        page = await context.new_page()
        await stealth_async(page)
        await fetch_offer(page, offer_url, args.raw_html_dir)
        await context.close()
```

Na końcu wywoływana jest równoległa grupa zadań:

```
await asyncio.gather(*[worker(url) for url in offer_urls])
```

2.3 Zbieranie linków ofert z listy wyników

Pierwszym krokiem jest pobranie linków do poszczególnych ofert ze stron listowych, takich jak <https://www.otodom.pl/pl/wyniki/sprzedaz/mieszkanie/cala-polska?page=1> Kod do wyodrębniania linków wygląda następująco:

```
def extract_offer_urls(html: str) -> List[str]:
    soup = BeautifulSoup(html, "html.parser")
    urls = set()
    for a in soup.find_all("a", href=True):
        href = a["href"]
        if "/pl/oferta/" in href:
            if href.startswith("/"):
                href = BASE + href
            urls.add(href.split("?")[0])
    return list(urls)
```

Dzięki temu najpierw budowana jest duża lista unikalnych adresów, a dopiero potem pracownicy (workery) pobierają ofertę po ofercie równolegle.

2.4 Mechanizmy obrony przed blokadą

Ponieważ Otodom posiada systemy wykrywające nietypowy ruch, konieczne było zaimplementowanie kilku zabezpieczeń, aby crawler działał stabilnie i nie był blokowany.

Zastosowane techniki:

1. Losowane User-Agenty

```
context = await browser.new_context(  
    user_agent=random.choice(UA_POOL)  
)
```

2. Tryb stealth

Ukrywa wskaźniki automatyzacji (np. `navigator.webdriver`).

```
await stealth_async(page)
```

3. Adaptive sleep

Drobne przerwy między żądaniami:

```
await asyncio.sleep(random.uniform(0.4, 1.2))
```

4. Limity równoległości

Aby nie przeciążyć serwera i nie wywołać blokad:

```
sem = asyncio.Semaphore(args.concurrency)
```

3 Przygotowanie danych

W ramach projektu wykorzystano dwa zbiory danych dotyczące rynku nieruchomości. Pierwszy z nich stanowi publicznie dostępny zbiór z platformy Kaggle, zawierający informacje o cenach mieszkań w Indiach, który posłużył do wstępnych eksperymentów. Drugi zbiór został pozyskany przez scraping portalu OtoDom i stanowi dane docelowe projektu.

3.1 Zbiór danych z Kaggle

Zbiór danych z Kaggle posłużył do wstępnych testów różnych metod przygotowania danych oraz eksperymentów z modelami. Pierwotnie zawierał on ponad 187 tysięcy rekordów oraz 21 kolumn opisujących różne cechy mieszkań na indyjskim rynku nieruchomości.

Na początku usunięto kolumny, które nie niosły istotnych informacji lub zawierały wyłącznie braki danych, takie jak `Plot Area`, `Dimensions` czy `Description`. Kolejnym krokiem było usunięcie rekordów z brakami w najważniejszych zmiennych - `Bathroom`, `Floor` i `Price`, co zmniejszyło zbiór do około 162 tysięcy obserwacji.

Dla pozostałych braków zastosowano różne strategie uzupełniania. Dla zmiennych takich jak `Balcony` czy `Car Parking` przyjęto, że brak informacji oznacza brak danej cechy, więc uzupełniono je zerem. W przypadku `Carpet Area` obliczono średnią powierzchnię dla każdej liczby sypialni i na tej podstawie dokonano imputacji brakujących wartości.

Sporo pracy wymagało przekształcenie danych tekstowych na numeryczne. Na przykład z kolumny `Floor` zawierającej wartości typu "10 out of 11" czy "Ground out of 7" wyekstrahowano dwie zmienne: numer piętra i całkowitą liczbę pięter w budynku. Podobnie przetworzono informacje o powierzchni, usuwając jednostki "sqft" i konwertując na liczby. Dla miejsc parkingowych wyodrębniono liczbę miejsc, dodatkowo uwzględniając typ: miejsca kryte ważono współczynnikiem 1.3, a otwarte 1.0.

Zmienne kategoryczne zakodowano na różne sposoby. Dla `Furnishing` zastosowano kodowanie porządkowe (0 dla nieumeblowanych, 1 dla częściowo, 2 dla w pełni umeblowanych), a dla zmiennych takich jak `facing`, `overlooking` czy `Ownership` użyto one-hot encoding. Z tytułu oferty wyekstrahowano liczbę sypialni używając wyrażeń regularnych.

Na końcu dokonano podziału danych na zbiór treningowy i testowy w proporcji 80:20, otrzymując ostatecznie 29 zmiennych gotowych do modelowania.

3.2 Zbiór OtoDom

Zbiór zawiera rzeczywiste oferty sprzedaży mieszkań z polskiego rynku nieruchomości. Głównym celem było zebranie dużego zbioru danych bezpośrednio z Otodom.pl, który następnie mógłby posłużyć do trenowania modelu predykcyjnego.

Wyjściowy zbiór składał się z ponad 44 tysięcy ofert oraz 33 kolumn zawierających różnorodne informacje o mieszkaniach, od podstawowych cech takich jak powierzchnia i liczba pokoi, przez szczegóły konstrukcyjne budynku, aż po lokalizację geograficzną i dodatkowe udogodnienia.

Na początku usunięto kolumny, które nie były istotne dla modelowania lub zawierały informacje administracyjne, takie jak `ad_id` czy `date_created`. Usunięto również kolumny `floor`, `building_floors` oraz `floor_type`, które zawierały informacje już uwzględnione w innych zmiennych lub miały za dużo braków.

W przypadku zmiennej `rooms` wartość "more" (oznaczająca więcej niż 10 pokoi) zastąpiono liczbą 10. Dla zmiennej `building_material` połączono rzadkie kategorie - wartości "HYDRO-TON" i "WOOD" przekształcono na "OTHER", aby uniknąć zbyt dużej fragmentacji danych.

Wszystkie zmienne kategoryczne przekształcono za pomocą one-hot encoding. Dotyczyło to takich kolumn jak `building_material`. Przed kodowaniem braki danych w tych zmiennych uzupełniono wartością "unknown".

Szczególną uwagę poświęcono zmiennej `building_year`. Zamiast używać surowego roku budowy, utworzono kategorie wiekowe budynków: "prewar" (przed 1945), "communist" (1945-1989), "modern" (1990-2009), "new" (od 2010) oraz "unknown" dla braków danych. Następnie zastosowano one-hot encoding dla tych kategorii, a oryginalną kolumnę usunięto.

Po wstępnym przetworzeniu danych przeprowadzono analizę ważności cech za pomocą Random Forest. Model ten pozwolił zidentyfikować najistotniejsze zmienne dla predykcji ceny. Usunięto cechy o ważności poniżej 0.001, co zmniejszyło liczbę zmiennych z 57 do 37. Najważniejszymi cechami okazały się powierzchnia mieszkania (`area`), lokalizacja geograficzna (`lon`, `lat`) oraz obecność windy (`has_elevator`).

Na końcu dokonano podziału na zbiór treningowy i testowy w proporcji 80:20, otrzymując odpowiednio 35 258 i 8 815 rekordów.

4 Modelowanie

4.1 Sieć neuronowa

W pierwszej kolejności postanowiono zastosować wielowarstwową sieć neuronową (MLP - Multi-layer Perceptron) do predykcji cen mieszkań. Model został zaimplementowany z wykorzystaniem biblioteki TensorFlow/Keras.

4.1.1 Architektura sieci

Przetestowano dwie architektury sieci. Wariant 1 składał się z pięciu warstw gęstych o malejącej liczbie neuronów: 128, 64, 32, 16 i warstwy wyjściowej z jednym neuronem. Wariant 2 był głębszy i składał się z warstw: 256, 128, 64, 32 oraz warstwy wyjściowej. W obu przypadkach zastosowano aktywację ReLU dla warstw ukrytych.

Aby zapobiec przeuczeniu, wykorzystano techniki regularyzacji. W pierwszych dwóch warstwach wariantu 1 dodano regularyzację L2 o współczynniku 0.001. Każda warstwa ukryta (poza ostatnią) była normalizowana za pomocą Batch Normalization, a następnie zastosowano Dropout o wartościach od 0.2 do 0.35 w zależności od warstwy. Model kompilowano z optymalizatorem Adam o współczynniku uczenia 0.001 oraz funkcją straty Huber z parametrem delta równym 1.0, która jest odporna na wartości odstające.

4.1.2 Przygotowanie danych

Przed treningiem dane zostały przeskalowane za pomocą StandardScaler. Cechy wejściowe znormalizowano do średniej 0 i odchylenia standardowego 1. Dla zmiennej docelowej (ceny) zastosowano transformację logarytmiczną $\log(1 + x)$, co pozwoliło lepiej modelować rozkład cen mieszkań, który charakteryzuje się silną asymetrią.

Zbiór treningowy podzielono dodatkowo, wydzielając 15% danych na walidację. Trenowanie odbywało się z batch size równym 32 przez maksymalnie 300 epok.

4.1.3 Mechanizmy kontroli treningu

Zaimplementowano dwa callbacki kontrolujące proces uczenia. EarlyStopping monitorował metrykę MAE na zbiorze walidacyjnym i przerywał trening po 40 epokach bez poprawy, przywracając wagi z najlepszej epoki. ReduceLROnPlateau zmniejszał współczynnik uczenia o połowę po 12 epokach bez poprawy, z minimalną wartością 0.00001.

4.1.4 Wyniki

Po zakończeniu treningu model osiągnął średni błąd absolutny (MAE) na poziomie około 17% wartości rzeczywistej. Analiza błędów wykazała, że sieć neuronowa miała trudności z precyzyjną predykcją, szczególnie dla mieszkań o ekstremalnych cenach. Pomimo zastosowania zaawansowanych technik regularyzacji i optymalizacji hiperparametrów, wyniki nie były satysfakcjonujące. Z tego powodu zdecydowano się na przetestowanie alternatywnego podejścia - modelu gradient boosting w postaci CatBoost.

4.2 Model CatBoost

Ze względu na niezadowalające wyniki sieci neuronowej, zdecydowano się na zastosowanie algorytmu gradient boosting w postaci CatBoost. Jest to zaawansowana implementacja metody wzmacniania gradientowego, szczególnie efektywna w zadaniach regresji z danymi tabelarycznymi. CatBoost charakteryzuje się natywną obsługą zmiennych kategorycznych oraz wbudowanymi mechanizmami przeciwdziałającymi przeuczeniu.

4.2.1 Wstępna analiza i przygotowanie danych

Przed rozpoczęciem modelowania przeprowadzono szczegółową analizę rozkładu zmiennej docelowej. Zbiór treningowy zawierał 35 258 przykładów, a zbiór testowy 8 815 przykładów. Zakres cen wynosił od 3 500 PLN do 25 000 000 PLN, przy medianie 580 000 PLN i średniej 699 706 PLN. Znaczna różnica między medianą a średnią wskazywała na obecność obserwacji odstających w zbiorze danych.

W celu poprawy jakości modelowania zdecydowano się na usunięcie ekstremalnych wartości. Wykorzystując percentyle 1% (167 000 PLN) i 99% (2 850 000 PLN), wyeliminowano 702 obserwacje ze zbioru treningowego (2.0%) oraz 182 ze zbioru testowego (2.1%). Po oczyszczeniu danych zakres cen zawężono do przedziału 167 000–2 850 000 PLN, co pozwoliło na znaczące ograniczenie wpływu wartości odstających na proces uczenia modelu. Po tym etapie zbiór treningowy liczył 34 556 przykładów, a testowy 8 633 przykładów.

4.2.2 Architektura i parametry modelu

Model CatBoost został skonfigurowany z uwzględnieniem lekcji wyniesionych z wcześniejszych eksperymentów, w których zaobserwowano silne przeuczenie. Pierwotna konfiguracja z 20 000 iteracji, współczynnikiem uczenia 0.02 i głębokością drzew równą 9 prowadziła do ekstremalnego przeuczenia, gdzie stosunek RMSE między zbiorem testowym a treningowym wynosił 6.3 (RMSE train = 36 116 PLN vs RMSE test = 226 638 PLN).

W finalnej wersji zastosowano konserwatywne parametry oraz wzmocnione mechanizmy regularyzacji. Liczbę iteracji ograniczono do 3 000, przy jednoczesnym zwiększeniu współczynnika uczenia z 0.02 do 0.08, co pozwoliło na szybszą zbieżność przy mniejszej liczbie drzew. Najistotniejszą zmianą było zmniejszenie maksymalnej głębokości drzew z 9 do 5 poziomów, parametr ten ma bezpośredni wpływ na zdolność modelu do uchwycenia skomplikowanych interakcji, ale jednocześnie zwiększa ryzyko przeuczenia.

Regularyzację L2 dla liści wzmocniono pięciokrotnie, zwiększając parametr `l2_leaf_reg` z 3 do 15. Wprowadzono też parametry `bagging_temperature = 2` i `random_strength = 2`, które dodają stochastyczność do procesu budowania drzew. Subsampling danych ustawiono na poziomie 0.7, co znaczy, że każde drzewo było trenowane na losowym 70% podzbiorze danych treningowych.

Dodatkowo nałożono ograniczenia strukturalne na poszczególne drzewa: `min_data_in_leaf = 50` (minimalna liczba obserwacji w liściu) oraz `max_leaves = 32` (maksymalna liczba liści). Te parametry zapobiegają tworzeniu zbyt specjalistycznych liści dopasowanych do szumu w danych. Mechanizm wczesnego zatrzymania (`early_stopping_rounds`) ustawiono na 100 iteracji, co pozwalało na przerwanie treningu, gdy model przestawał poprawiać wyniki na zbiorze walidacyjnym. W praktyce model wykorzystał wszystkie 3 000 iteracji, co sugeruje, że proces uczenia przebiegał stabilnie bez oznak degradacji wyników.

4.2.3 Walidacja krzyżowa

Przed trenowaniem finalnego modelu przeprowadzono 5-krotną walidację krzyżową w celu oceny stabilności i zdolności generalizacji. Wyniki CV wykazały średni RMSE na poziomie 130 799 PLN z odchyleniem standardowym 3 822 PLN, co świadczy o dobrej stabilności modelu. Względnie małe odchylenie standardowe (około 3% wartości średniej) potwierdza, że model nie jest nadmiernie wrażliwy na konkretny podział danych. Wynik walidacji krzyżowej był spójny z wynikiem na zbiorze testowym (127 842 PLN), co dodatkowo potwierdza, że model dobrze generalizuje na nowe dane.

4.2.4 Wyniki i analiza

Finalny model został wytrenowany na pełnym zbiorze treningowym. Na zbiorze treningowym osiągnięto $MAE = 67\,483$ PLN, $RMSE = 102\,112$ PLN oraz $MAPE = 10.66\%$. Na zbiorze testowym metryki wyniosły odpowiednio: $MAE = 78\,261$ PLN, $RMSE = 127\,842$ PLN oraz $MAPE = 12.11\%$. Współczynnik determinacji R^2 osiągnął wartość 0.8804, co oznacza, że model 88% różnic potrafi przewidzieć na podstawie cech.

Stosunek między wynikami na zbiorze treningowym a testowym (test/train) dla RMSE wyniósł 1.252, co klasyfikuje się jako umiarkowane przeuczenie. Stanowi to jednak dramatyczną poprawę w stosunku do początkowej wersji modelu, gdzie stosunek wynosił 6.3. Podobnie, stosunek dla MAE wyniósł 1.160, co również potwierdza znaczące ograniczenie problemu przeuczenia.

Szczegółowa analiza błędów wykazała, że 55.2% predykcji charakteryzuje się błędem procentowym poniżej 10%, a 72.7% poniżej 15%. W ujęciu absolutnym, 76.5% predykcji mieści się w przedziale błędu $\pm 100\,000$ PLN, co stanowi zadowalający wynik dla rynku nieruchomości charakteryzującego się wysoką zmiennością cen. Mediana błędu absolutnego wyniosła 49 169 PLN, co przy medianie cen równej 580 000 PLN przekłada się na około 8.5% błędu względnego dla typowego mieszkania.

Analiza przypadków skrajnych ujawniła, że najlepsze predykcje charakteryzowały się błędem poniżej 1% (minimum 26 PLN), podczas gdy najgorsze przypadki dotyczyły mieszkań o nietypowych charakterystykach lub lokalizacjach, gdzie błąd mógł przekraczać 1 400 000 PLN. Niemniej jednak, takie przypadki stanowiły mniej niż 0.1% wszystkich predykcji.

4.2.5 Analiza ważności cech

Mechanizm feature importance w CatBoost pozwolił zidentyfikować najistotniejsze zmienne dla predykcji ceny. Bezwzględnie najważniejszą cechą okazała się **area** (powierzchnia mieszkania) z wagą 28.2%, co potwierdza intuicyjne założenie o kluczowym znaczeniu wielkości nieruchomości dla jej wartości. Na drugim miejscu znalazła się **lon** (długość geograficzna) z 26.6%, a na trzecim **lat** (szerokość geograficzna) z 21.4%.

Łącznie współrzędne geograficzne (**lat** i **lon**) odpowiadają za około 48% całkowitej ważności, co jednoznacznie wskazuje, że lokalizacja jest czynnikiem równie istotnym jak powierzchnia mieszkania. Ten wynik potwierdza znaną zasadę rynku nieruchomości, że trzy najważniejsze czynniki to: "lokalizacja, lokalizacja i lokalizacja".

Spośród pozostałych cech istotny wpływ miała obecność windy (`has_elevator`) z wagą 7.5%, co szczególnie istotne jest dla mieszkań na wyższych piętrach. Typ okien (`windows_type_PLASTIC`) oraz typ budynku (`building_type_APARTMENT`, `building_type_BLOCK`) również znalazły się w czołówce najważniejszych zmiennych, choć ich łączny wpływ był znacznie mniejszy (łącznie około 5%).

Warto zauważyć, że model wykorzystał wyłącznie oryginalne cechy ze zbioru danych, bez dodatkowych przekształceń czy inżynierii cech. Świadczy to o zdolności algorytmu CatBoost do automatycznego wychwytywania nieliniowych zależności i interakcji między zmiennymi, co jest jedną z kluczowych zalet tej metody.

4.3 Adaptacja modelu CatBoost dla danych z Kaggle

Wobec dobrych rezultatów uzyskanych na polskich danych z Otodom, podjęto decyzję o weryfikacji działania modelu na alternatywnym zbiorze pochodzącym z platformy Kaggle, dotyczącym rynku nieruchomości w Indiach. Zastosowanie modelu do danych o odmiennej charakterystyce wymagało wprowadzenia modyfikacji w pipeline’ie przetwarzania.

4.3.1 Zmiany w przygotowaniu danych

Najistotniejsza różnica dotyczyła skalowania zmiennej docelowej. O ile ceny w zbiorze Otodom wyrażone były w złotych polskich i oscylowały wokół mediany 580 000 PLN, dane z Kaggle zawierały ceny w rupiach indyjskich o medianie zaledwie 6 004 INR. Tak znacząca różnica w skalach wartości skłoniła do zastosowania transformacji logarytmicznej, która znormalizowała rozkład i umożliwiła modelowi lepsze uczenie się na względnych zmianach cen.

Strategia czyszczenia wartości odstających została zmodyfikowana - zamiast odrzucania obserwacji poniżej 1. i powyżej 99. percentyla, zastosowano bardziej selektywne podejście z progami na poziomie 0.1% i 99.5%. W praktyce wyeliminowano jedynie około 0.6% rekordów, głównie te zawierające ceny zerowe lub ujemne, będące prawdopodobnie błędnym zapisem.

4.3.2 Rekonfiguracja hiperparametrów

Zmiany objęły również kluczowe parametry modelu. Liczbę iteracji podniesiono do 4 000 (wobec wcześniejszych 3 000), co było uzasadnione stabilnym procesem uczenia obserwowanym w logach treningowych, model wykorzystał wszystkie iteracje bez aktywacji mechanizmu wczesnego zatrzymania. Współczynnik uczenia zredukowano nieznacznie do 0.025 (z 0.08), co w połączeniu z większą liczbą kroków zapewniło płynniejszą zbieżność.

Regularyzację dostosowano przez zmniejszenie `12_leaf_reg` z 15 do 12 oraz zwiększenie subsamplingu z 0.7 do 0.8. Paradoksalnie, pozwolono również na głębsze drzewa (głębokość 6), co okazało się nie prowadzić do przeuczenia dzięki odpowiednio dobranym mechanizmom regularyzacyjnym. Parametr `min_data_in_leaf` został obniżony z 50 do 40 obserwacji.

4.3.3 Analiza osiągniętych wyników

Finalny model wykazał metryki zbliżone do tych uzyskanych na danych polskich. Współczynnik R^2 osiągnął wartość 0.877 na zbiorze testowym, przy MAPE równym 10.13%. Co szczególnie istotne, stosunek RMSE między zbiorem testowym a treningowym wyniósł zaledwie 1.074, co jednoznacznie wskazuje na brak problemu przeuczenia. Model demonstrował stabilne zachowanie: 72.3% predykcji charakteryzowało się błędem poniżej 10%, zaś 82.1% mieściło się w przedziale $\pm 1\,000$ rupii.

W hierarchii ważności cech dominującą pozycję zajęła zakodowana lokalizacja (`location_encoded`) z wagą 38.98%, co ponownie potwierdza fundamentalną rolę położenia geograficznego w wycenie nieruchomości. W odróżnieniu jednak od danych polskich, gdzie współrzędne geograficzne dominowały z łączną wagą 48%, w danych indyjskich większe znaczenie zyskały parametry konstrukcyjne budynku – liczba pięter (`TotalFloors`, 15.45%) oraz powierzchnia użytkowa (`Carpet Area`, 8.22%). Należy jednak zauważyć, że w zbiorze polskim informacje o liczbie pięter charakteryzowały się bardzo wysokim odsetkiem brakujących danych, co uniemożliwiło pełne wykorzystanie tej cechy. Prawdopodobnie przy kompletnych danych parametry konstrukcyjne odgrywałyby podobnie istotną rolę również w polskim kontekście.

5 Aplikacja webowa

W ramach rozwiązania przewidziano stworzenie interfejsu webowego w celu dostępu użytkownika do prostego użycia modelu w prawdziwych warunkach. Aplikacja została stworzona z użyciem frameworku Vue.js z Vite w języku TypeScript.

W ramach aplikacji stworzone zostały dwa oddzielne widoki dla obydwóch realizowanych ścieżek (Polska i Indie):

1. Widok mapy
2. Widok szczegółowy

Widok mapy oferuje predykcje ceny na podstawie wybrania lokalizacji z mapy w wersji polskiej, lub jednego z 81 miast w wersji indyjskiej.

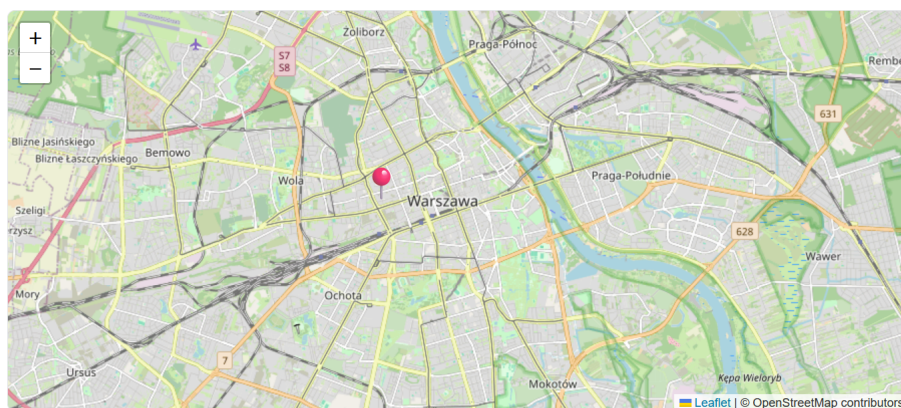
5.1 Widok mapy/lokalizacji

5.1.1 Polska

W tym widoku z pomocą biblioteki **Leaflet** umożliwiającej importowanie map do aplikacji webowych została dodana mapa, na której użytkownik może wybrać dowolny punkt poprzez zaznaczenie pinezką. Współrzędne geograficzne zaznaczonego punktu są automatycznie ładowane do formularza dostępnego w tym widoku.

Wycena oparta na lokalizacji (Polska)

Wybierz lokalizację na mapie lub wprowadź współrzędne. Dodaj parametry nieruchomości dla dokładniejszej wyceny.



Rysunek 1: Widok mapy w wersji modelu z polskimi danymi

Po wypełnieniu pól formularza oraz kliknięciu przycisku **Wycena nieruchomości**, wysyłany jest request do serwera, który zwraca cenę przewidzianą przez model

5.1.2 Indie

Widok ten umożliwia wybór miasta, w którym znajduje się mieszkanie. Zbiór zawiera 81 miast do wyboru.

Wycena oparta na mieście (Indie)

Wybierz miasto i wprowadź parametry nieruchomości, aby uzyskać przewidywaną cenę.

Miasto
Bangalore

Powierzchnia (stopy kw.)
50

Liczba pokoi
2

Liczba łazienek
2

Wycena nieruchomości

Rysunek 2: Widok miast w wersji modelu z indyjskimi danymi

Ponownie, po wypełnieniu pól i wysłaniu zapytania, model trenowany na danych indyjskich zwróci odpowiednią, przewidywaną cenę.

5.2 Widok szczegółowy

Widok szczegółowy daje użytkownikowi możliwość dokładnego wyboru specyfikacji nieruchomości, oraz wprowadzenie prawdziwej ceny (np. z ogłoszenia).

5.2.1 Polska

Szczegółowa analiza nieruchomości

Wprowadź pełne dane nieruchomości dla precyzyjnej wyceny AI. Dodaj cenę wywoławczą aby sprawdzić czy oferta jest uczciwa.

Wybierz miasto

Warszawa Kraków Wrocław Gdańsk Poznań

Szerokość geograficzna * 52.2297 Długość geograficzna * 21.0122 Powierzchnia (m²) * 50 Liczba pokoi * 2

Rok budowy 2010 Materiał budynku Cegła Forma własności Własność Stan wykończenia Do zamieszkania

Typ ogrzewania Miejskie Typ budynku Blok Typ okien Plastikowe Cena wywoławcza (PLN) Opcjonalnie - do analizy

Wypożażenie i udogodnienia

☒ Balkon ☐ Parking ☐ Taras ☐ Ogród ☐ Piwnica

☒ Winda ☐ Ochrona ☒ Kuchenska ☒ Piekarnik ☐ Lodówka

☐ Zmywarka

Wycena nieruchomości

Rysunek 3: Widok szczegółowy w wersji modelu z polskimi danymi

W przypadku wypełnienia prawdziwej ceny nieruchomości, dodatkowo do przewidywanej ceny zostanie dodany fragment o tym, czy mieszkanie jest poniżej ceny rynkowej, w odpowiedniej cenie lub czy jest po zawyżonej cenie.

5.3 Serwer API

W celu użycia modelu w aplikacji webowej, konieczne było stworzenie serwera API, który umożliwia wywołanie modelu na wprowadzonych danych.

Stworzono następujące endpointy:

1. **http://localhost:5000/** – informacje o API
2. **http://localhost:5000/health** – Health check
3. **http://localhost:5000/cities** – Zwraca listę dostępnych miast dla Indii. Zwracany format danych:

```
{
  "cities": ["agra", "ahmadnagar", "ahmedabad", "bangalore", "..."]
}
```

4. **http://localhost:5000/predict/poland** – Zwraca predykcje w postaci:

```
{
  "predictedPrice": 5000000.00,
  "confidence": 0.85
}
```

Przykładowy format danych wejściowych:

```
{
  "latitude": 52.2297,
  "longitude": 21.0122,
  "area": 50,
  "rooms": 2,
  "yearBuilt": 2010,
  "buildingMaterial": "BRICK",
  "ownership": "FULL_OWNERSHIP",
  "condition": "READY_TO_USE",
  "heatingType": "URBAN",
  "buildingType": "BLOCK",
  "windowsType": "PLASTIC",
  "hasBalcony": true,
  "hasElevator": true
}
```

5. **http://localhost:5000/predict/poland** – Zwraca przewidzianą cenę dla modelu indyjskiego.

Przykładowe dane wejściowe:

```
{  
  "city": "bangalore",  
  "area": 1000,  
  "rooms": 2,  
  "bathrooms": 2,  
  "hasBalcony": true,  
  "hasParking": true  
}
```

6 Wnioski

W trakcie pracy zebrano dane, zbudowano model, API oraz interfejs webowy w celu użycia modelu przez użytkownika.

Przez zaznaczenie miejsca na mapie pinezką możliwe było uzyskanie dokładnych danych lokalizacyjnych, co z kolei umożliwiło stosunkowo dokładną predykcję ceny mieszkania.

Finalny produkt w formie interaktywnej mapy celnie określał przewidywane ceny, podobne to takich jakie mogą być spodziewane w rzeczywistości. Jedyna niezgodność występowała w przypadku różnic pomiędzy dzielnicami miast. W celu rozwiązania tego problemu, trzeba by było trenować osobne modele na zbiorach danych o podobnej wielkości ale tylko z konkretnych miast. Niestety w przypadku trenowania na danych z całego kraju, model jest w stanie trafnie przewidzieć cenę jedynie biorąc pod uwagę odległość mieszkania od centrum.

Biblioteka Leaflet użyta do wyświetlenia mapy z możliwością zaznaczenia dowolnej lokalizacji pinezką okazała się kluczowa, przez fakt, że nie wymaga żadnej opłaty od użytkownika. Spełniła swoje zadanie umożliwiając pobranie współrzędnych dowolnie wybranego miejsca na mapie.